

# Sztuczna inteligencja. A\* (cd). Problemy więzowe

Paweł Rychlikowski

Instytut Informatyki UWr

25 marca 2026

# Komandos z mapą. Mniej trywialny przykład

- Rozważmy zadanie, w którym do labiryntu wrzucony zostaje komandos z mapą...
- ale zrzut jest w nocy i nie wiadomo, gdzie trafił.
- Problem:  
*znajdź sekwencję akcji, która **na pewno** doprowadzi do jednego z celów (akcje niedozwolone nie przesuwają komandosa).*

# Komandos. Jak go rozwiązać

- Zadanie z komandosem będzie na liście P2.
- Trochę podobne do zadanie 6 z C1 (z lidarem), ale bez lidaru.
- Zbadajmy na tablicy, jak działa taka przestrzeń przekonań.

## Część 1

Przeszukiwanie z z wiedzą o problemie (informed search).  
Dokończenie

# Algorytm A\*. Przypomnienie

## Definicje

- $g(n)$  – koszt dotarcia do węzła  $n$
- $h(n)$  – szacowany koszt dotarcia od  $n$  do (najbliższego) punktu docelowego ( $h(s) \geq 0$ )
- $f(n) = g(n) + h(n)$

## Algorytm

Przeprowadź przeszukiwanie, wykorzystując  $f(n)$  jako priorytet węzła (czyli rozwijamy węzły od tego, który ma najmniejszy  $f$ ).

# Punkty orientacyjne

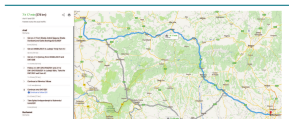


Figure 3.28 A Web service providing driving directions, computed by a search algorithm.

## Landmarks

Zakładamy istnienie pewnej liczby **punktów orientacyjnych**, będziemy się do nich odwoływać przy liczeniu heurystycznej odległości między węzłami

Zakładamy, że policzyliśmy odległości między **każdym p.o.** a **każdym** węzłem.

## Heurystyka „naturalna”

$$h(n) = \min_{L \in \text{Landmarks}} (C^*(n, L) + C^*(L, \text{goal}))$$

Ogólnie nie jest optymistyczna, chyba że ścieżka przechodzi przez *jakiś* punkt orientacyjny!

## Differential heuristic

$$h(n) = \max_{L \in \text{Landmarks}} (C^*(n, L) - C^*(L, \text{goal}))$$

(jak wyjdzie ujemna, to dajemy 0)

- Myślimy, że punkt orientacyjny jest **za celem** (jadąc do L mijamy cel po drodze)
- Jak cel jest trochę z boku drogi, to tracimy dokładność, ale nie optymizm.

# Heurystyki niedopuszczalne

- Heurystyki mogą być niedopuszczalne (w szczególności, jeżeli są wynikiem uczenia się heurystyk)
- Oczywiście tracimy wówczas (w teorii i praktyce) gwarancje optymalności.
- Ale można otrzymać istotnie szybsze wyszukiwanie (o czym, mam nadzieję, przekonamy się na pracowni 2)



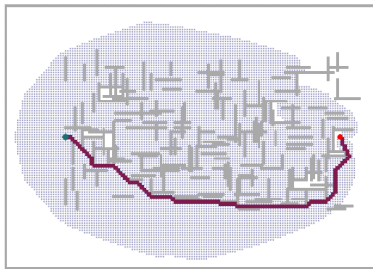
## Weighted $A^*$ search

$$f(n) = g(n) + W \times h(n), \text{ dla } W > 1$$

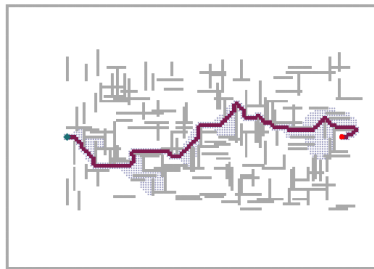
Różne warianty  $W$ , dla podsumowania

- $A^*$ :  $g(n) + h(n)$ , czyli  $W = 1$
- **Uniform-cost search**:  $g(n)$ , czyli  $W = 0$
- **Greedy best-first search**:  $h(n)$ , czyli  $W = \infty$
- **Weighted  $A^*$  search**

# Przykładowe wyniki Weighted A\*



(a)



(b)

**Figure 3.21** Two searches on the same grid: (a) an A\* search and (b) a weighted A\* search with weight  $W = 2$ . The gray bars are obstacles, the purple line is the path from the green start to red goal, and the small dots are states that were reached by each search. On this particular problem, weighted A\* explores 7 times fewer states and finds a path that is 5% more costly.

## Plan

Spróbujemy dowieść następujących rzeczy:

- 1)  $A^*$  zwraca najkrótszą drogę
- 2)  $A^*$  jest zupełny.

# Dowód optymalności

Potrzebujemy dwóch faktów:

- F1. Jeżeli  $h$  jest spójna, wówczas na każdej ścieżce wartości  $f$  są niemalejące.

D-d ( $n'$  jest następnikiem  $n$ ):

$$f(n') = g(n') + h(n') = g(n) + c(n, n') + h(n') \geq g(n) + h(n) = f(n)$$

- F2. Zawsze, gdy algorytm bierze węzeł do rozwinięcia, to koszt dotarcia do tego węzła jest optymalny (najmniejszy możliwy).

# Dowód F2



# Dowód F2



# Dowód F2

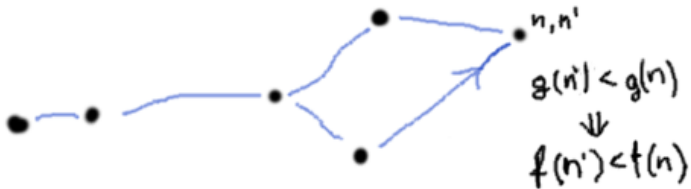


# Dowód F2

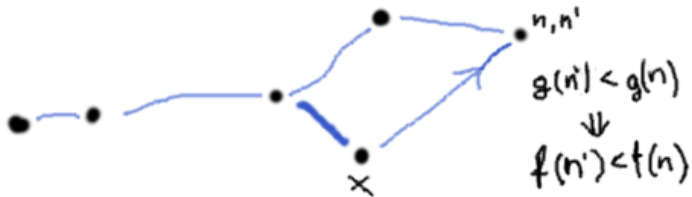




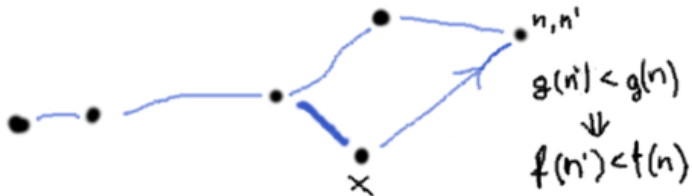
# Dowód F2



# Dowód F2



$$\text{z F1: } f(x) \leq f(n') < f(n)$$



$$\text{z F1: } f(x) \leq f(n') < f(n)$$

**SPRZECZNOŚĆ**

- Bierzemy nie wprost węzeł  $n$ , do którego kolejne dotarcie daje mniejszy koszt niż dotarcie pierwsze.
- Wartość  $f$  dla tego węzła drugi raz jest mniejsza ( $h$  takie same,  $g$  mniejsze)
- Na ścieżce od początku do  $n'$  drugiego mamy widziany w pierwszym przebiegu węzeł  $x$  (tuż po rozgałęzieniu)
- Z własności F1 mamy:  $f(x) < f(n)$

Zatem algorytm powinien wybrać  $x = x_1$  przed  $n$ .

Podobnie  $x_2, x_3, \dots$  **Sprzeczność.**

Popatrzmy na pierwszy znaleziony węzeł docelowy ( $n_{\text{end}}$ )

- $f(n_{\text{end}}) = g(n_{\text{end}}) + h(n_{\text{end}}) = g(n_{\text{end}})$  (bo  $h$  jest rozsądna)
- Każdy kolejny węzeł docelowy jest nielepszy, bo dla niego  
 $f(n) \geq f(n_{\text{end}})$   
(czyli  $g(n) \geq g(n_{\text{end}})$ , bo  $h$  dla końcowych jest 0)

Niech  $C^*$  będzie kosztem najtańszego rozwiązania ( $g(n_{\text{end}})$ )

- Algorytm ogląda wszystkie węzły, dla których  $f(n) < C^*$
- Być może oglądnie również pewne węzły z konturu  $f(n) = C^*$ , przed wybraniem docelowego  $n$ , t.ż.  $f(n) = g(n) + 0 = C^*$

## Uwaga

Skończona liczba węzłów o  $g(n) \leq C^*$  gwarantuje to, że algorytm się skończy.

Do skończoności z kolei wystarczy na przykład założyć, że

- istnieje  $\varepsilon > 0$ , t.ż. wszystkie koszty są od niego większe bądź równe,
- węzły mają skończoną liczbę dzieci.

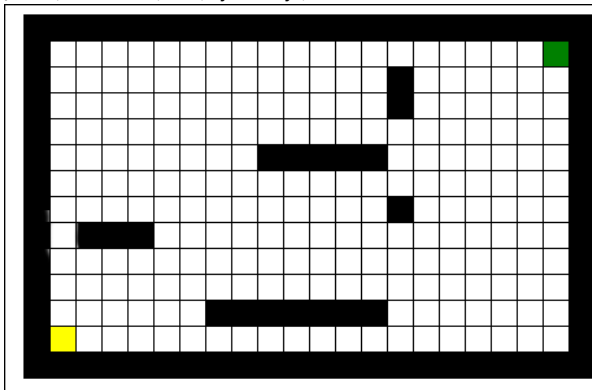
## Uwaga

$A^*$  nie rozwija węzłów t.ż.  $f(n) > C^*$ . Zatem im większa  $h$  (przy założeniu spełniania warunków dobrej heurystyki), tym lepsza.

## Pytanie

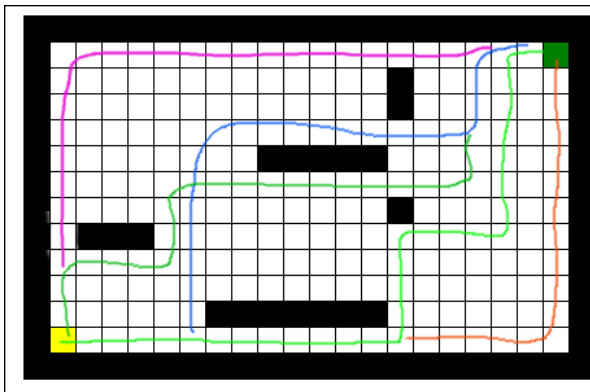
Co się będzie działo, jeżeli nasza funkcja  $h$  będzie liczyła **dokładną** odległość od celu?

Bierzemy heurystykę Manhatańską (przyjmijmy, że cel jest jeden),  
czyli  $h(n) = |g_x - n_x| + |g_y - n_y|$





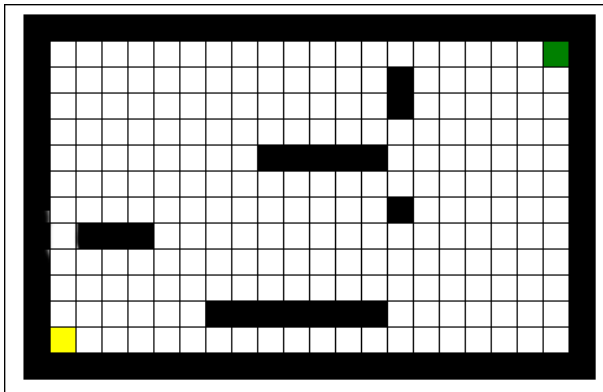
# Płaska funkcja $f$



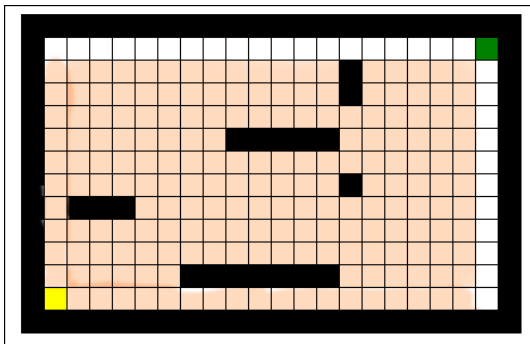
Wszystkie ścieżki idące w prawo i do góry są optymalne. Funkcja  $f$  jest stała (równa 30).

# Porównanie heurystyk

Porównajmy na przykładzie heurystykę manhatańską i euklidesową.  
Które węzły **na pewno** musi obejrzeć  $A^*$  z heurystyką Euklidesową?



# Uzasadnienie lepszości heurystyki manhattańskiej



- Funkcja  $f$  z heurystyką  $h_M$  ma wszędzie wartość 30
- Funkcja  $f$  z heurystyką  $h_E$  osiąga wartość 30 jedynie na brzegach.

## Algorytm

Przeprowadź przeszukiwanie, wykorzystując  $f(n)$  jako priorytet wężła (czyli rozwijamy węzły od tego, który ma najmniejszy  $f$ ).

## Kluczowa właściwość

1.  $A^*$  rozwija wszystkie węzły t.ż.  $f(n) < C^*$ .
2.  $A^*$  rozwija niektóre węzły t.ż.  $f(n) = C^*$
3.  $A^*$  nie rozwija węzłów t.ż.  $f(n) > C^*$ .

## Część 2

### Problemy więzowe (CSP = Constraint Satisfaction Problem)

## Uwaga

Między **ósemką** a **hetmanami** jest istotna różnica (mimo, że oba można przedstawiać jako problemy przeszukiwania).

- W ósemce interesuje nas droga dotarcia do celu, który jest dobrze znany (i tym samym mało ciekawy)
- W hetmanach interesuje nas, jak wygląda cel – droga do niego może być dość trywialna (dostawianie po kolei poprawnych hetmanów, przestawianie hetmanów z losowego ustawienia).

# Problem spełnialności więzów (2)

Problemy takie jak hetmany są:

- a) Bardzo istotne (ze względu na ich występowanie w rzeczywistym świecie)
- b) Na tyle specyficzne, że warto dla nich rozważać specjalne metody.

# Problemy spełnialności więzów. Definicja

## Definicja

Problem spełnialności więzów ma 3 komponenty:

1. Zbiór zmiennych  $X_1, \dots, X_n$
2. Zbiór dziedzin (przypisanych zmiennym)
3. Zbiór więzów, opisujących dozwolone kombinacje wartości jakie mogą przyjmować zmienne.

## Przykład

**Zmienne:**  $X, Y, Z$

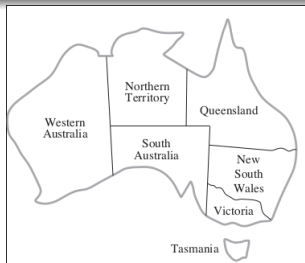
**Dziedziny:**  $X \in \{1, 2, 3, 4\}, Y \in \{1, 2\}, Z \in \{4, 5, 6, 7\}$

**Więzy:**  $X + Y \geq Z, X \neq Y$



- Powyższy przykład to były **więzy na dziedzinach skończonych**, jeden z najważniejszych przypadków więzów.
- Ale można rozważać inne dziedziny:
  - a) liczby naturalne, (trochę boli, że to **nierozstrzygalny problem**)
  - b) liczby wymierne,
  - c) ciągi elementów, napisy
  - d) krotki
- Więzy określają relacje, często da się je wyrazić wzorem, ale nie jest to wymagane.

# Kolorowanie Australii (inny przykład zadania więzowego)



- Mamy pokolorować mapę Australii, za pomocą 3 kolorów: (R, G, B)
- Sąsiadujące prowincje muszą mieć różne kolory.

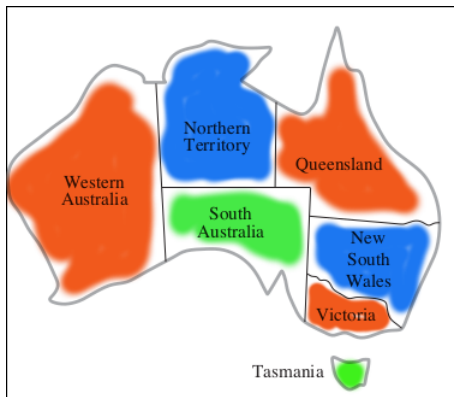
## Kolorowanie jako problem więzowy

**Zmienne:** WA, NT, Q, NSW, V, SA, T

**Dziedziny:** {R,G,B}

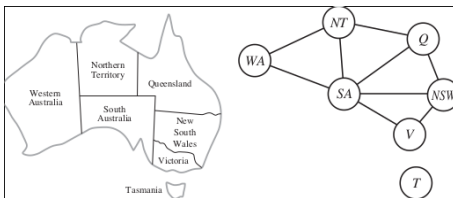
**Więzy:**  $SA \neq WA$ ,  $SA \neq NT$ ,  $SA \neq Q$ ,  $SA \neq NSW$ ,  $SA \neq V$ ,  $WA \neq NT$ ,  $NT \neq Q$ ,  $Q \neq NSW$ ,  $NSW \neq V$

# Przykładowe kolorowanie



- Więzy (jako relacje) mogą mieć różną arność.
- **Unarne** – można potraktować jako modyfikację dziedziny (Tasmania nie jest czerwona) i zapomnieć.
- **Binarne** – jak w naszym przykładzie z kolorowaniem
- Mogą mieć też inną arność, w zasadzie dowolną (w praktyce spotyka się więzy o arności np. kilkaset)

- Dla więzów **binarnych** możemy stworzyć graf, w którym krawędź oznacza, że dwie zmienne są powiązane więzem.
- Więzy binarne są istotną klasą więzów, wiele algorytmów działa przy założeniu binarności więzów.



# Problemy dualne

**Poziomo:**

1. Udzielana poszkodowanemu
4. Sprząta w szkole
7. Zawijasy
8. Tartaczny odpad
9. Leopold Staff
10. Przodek bydła domowego
11. Część drzewa
12. Schodzi z niego sztuka
13. Biblijny utwór poetycki
16. Zespół kojarzony z M. Grechutą
20. Ochronny rękaw
21. Partnerka
22. Właściciel gospodarstwa na Podhalu
23. Pola, serialowa Marusia

**Pionowo:**

1. Rów łączący np. dwa jeziora
2. Utopia
3. Miejsce, w którym coś się koncentruje
4. Ekspedycja badawcza
5. Otwór w tęczówce
6. Chustka zawiązana pod szyją
14. Daw. brak karności; niesforność
15. Inaczej lampart
17. Imię Żylińskiej, b. piosenkarki
18. Reklamowany pokarm dla kotów
19. Ferenc, słynny węg. kompozytor

## Pytanie

Co powinno być zmienną w **zadaniu rozwiązywania krzyżówki?**

# Krzyżówka (podstawowa)

Pomijamy (chwilowo?) kwestie zgodności hasła z definicją.



- Zmienne odpowiadają kratkom, dziedziną są znaki
- Wieży (fragment):  
jest-słowem-7(A,B,C,D,E,F,G), jest-słowem-3(B,H,I), ...

# Krzyżówka (dualna)

- Zmienne to słowa (dziedziną jest słownik przycięty do określonej długości)
- Mamy więc dla każdej pary krzyżujących się słów. Jaki?



Przykładowy więz  $C(w_1, w_2)$ :

- $w_1$  ma długość 6 (diedzina)
- $w_2$  ma długość 10 (diedzina)
- trzeci znak  $w_1$  jest taki sam, jak piąty znak  $w_2$

## Problemy dualne (2)

- Zwróćmy uwagę, że to, co zrobiliśmy z krzyżówką stosuje się do dowolnych więzów.
- Więzy w problemie prymarnym zmieniają się na zmienne w problemie dualnym (z dziedziną będącą dozwolonym zbiorem krotek)
- Dodatkowo potrzebujemy więzów, które mówią, że  $i$ -ty element jednej krotki jest  $j$ -tym elementem drugiej (te więzy są binarne!)

## Uwaga 1

Tym, co odróżnia CSP od zadania przeszukiwania, jest możliwość wykorzystania dodatkowej wiedzy o charakterze problemu do przeprowadzenia wnioskowania.

## Uwaga 2

Podstawowym celem wnioskowania jest zmniejszenie rozmiaru dziedzin (a tym samym zmniejszenie przestrzeni przeszukiwań).

# Wnioskowanie. Przykład

## Przykład

Zmienne:  $X, Y, Z$

Dziedziny:  $X \in \{1, 2, 3, 4\}, Y \in \{1, 2\}, Z \in \{5, 6, 7, 8\}$

Więzy:  $X + Y \geq Z, X \neq Y$

Czy możemy nie tracąc żadnego rozwiązania **skreślić** jakieś wartości z dziedzin?

Możemy wywnioskować, że:  $X \in \{3, 4\}, Y \in \{1, 2\}, Z \in \{5, 6\}$

Jak widać, możemy też skreślić więź  $X \neq Y$

- Spójność (intuicyjnie) rozumiemy jako **niemożność wykreślenia żadnej wartości z dziedziny**.
- Mamy różne rodzaje spójności:
  1. **Węzłowa** (każda wartość z dziedziny spełnia więzy unarne dla zmiennych)
  2. **Łukowa**: jak dwie zmienne są połączone węzem, to dla każdej wartości z dziedziny X jest wartość w dziedzinie Y, t.ż. dla tych wartości węz jest spełniony.

## Uwaga

Są jeszcze inne rodzaje spójności. Więcej na ćwiczeniach.

# Spójność więzów. Przykład

**Więź:**  $X < Y$

**Dziedzina X:**  $\{4, 6, 7, 10, 20\}$

**Dziedzina Y:**  $\{1, 2, 4, 6, 7, 10\}$

**Brak spójności**

- Jeżeli weźmiemy  $X$ , to możemy wykreślić wartości 10, 20
- Jeżeli weźmiemy  $Y$ , to możemy wykreślić wartości 1, 2, 4

Po wykreśleniu tych wartości warto przyjrzeć się innym więzom z  $X$  i  $Y$ .

## Definicja

Więź  $C$  dla zmiennych  $X$  i  $Y$  z dziedzinami  $D_X$  i  $D_Y$  jest **spójny łukowo**, wtt:

- Dla każdego  $x \in D_X$  istnieje takie  $y \in D_Y$ , że  $C(x, y)$  jest spełnione
- Dla każdego  $y \in D_Y$  istnieje takie  $x \in D_X$ , że  $C(x, y)$  jest spełnione

## Definicja

Więź  $C$  dla zmiennych  $X$  i  $Y$  z dziedzinami  $D_X$  i  $D_Y$  jest **spójny łukowo**, wtt:

- Dla każdego  $x \in D_X$  istnieje takie  $y \in D_Y$ , że  $C(x, y)$  jest spełnione
- Dla każdego  $y \in D_Y$  istnieje takie  $x \in D_X$ , że  $C(x, y)$  jest spełnione

**Sieć więzów** jest spójna łukowo, jeżeli każdy więź jest spójny łukowo.



Algorytm zapewnia spójność łukową sieci więzów.

## Idea

1. Zarządzamy kolejką więzów,
2. Usuujemy niepasujące wartości z dziedzin, analizując kolejne więzy z kolejki,
3. Po usunięciu wartości z dziedziny  $B$ , sprawdzamy wszystkie zmienne  $X$ , które występują w jednym więzie z  $B$

**function** AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise

**inputs:** *csp*, a binary CSP with components ( $X$ ,  $D$ ,  $C$ )

**local variables:** *queue*, a queue of arcs, initially all the arcs in *csp*

**while** *queue* is not empty **do**

    ( $X_i$ ,  $X_j$ )  $\leftarrow$  REMOVE-FIRST(*queue*)

**if** REVISE(*csp*,  $X_i$ ,  $X_j$ ) **then**

**if** size of  $D_i$  = 0 **then return** false

**for each**  $X_k$  **in**  $X_i$ .NEIGHBORS -  $\{X_j\}$  **do**

            add ( $X_k$ ,  $X_i$ ) to *queue*

**return** true

**function** REVISE(*csp*,  $X_i$ ,  $X_j$ ) **returns** true iff we revise the domain of  $X_i$

*revised*  $\leftarrow$  false

**for each**  $x$  **in**  $D_i$  **do**

**if** no value  $y$  in  $D_j$  allows ( $x, y$ ) to satisfy the constraint between  $X_i$  and  $X_j$  **then**

            delete  $x$  from  $D_i$

*revised*  $\leftarrow$  true

**return** *revised*

- Zwróćmy uwagę na niesymetryczność funkcji Revise (oznacza ona konieczność dodawania każdej pary zmiennych dwukrotnie)
- Zwróćmy uwagę, że istotny jest efekt uboczny tej funkcji: zmieniają się wartości dziedzin!

# Złożoność algorytmu AC-3

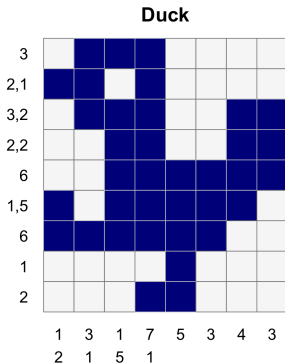
- Mamy  $n$  zmiennych, dziedziny mają wielkość  $O(d)$ . Mamy  $c$  więzów.
- Obsługa więzu to  $O(d^2)$
- Każdy więz może być włożony do kolejki co najwyżej  $O(d)$  razy.

## Złożoność

Złożoność wynosi zatem  $O(cd^3)$  (raczej pesymistycznie)

- Algorytm AC – 3 jest przykładowym algorytmem **propagacji więzów**
- Przeprowadzamy rozumowanie, które pozwala nam **bezpiecznie** usuwać zmienne z dziedziny.
- Zmniejszanie dziedziny zmiennej  $X$  może spowodować zmniejszenie dziedzin innych, związanych z nią zmiennych.

# Spójność i obrazki logiczne



- Zmienne to wiersze i kolumny
- Spójność węzłowa (pojedynczy wiersz/kolumna zgodna ze specyfikacją)
- Spójność łukowa – zmiany w dziedzinie wierszu wpływają na kolumny i vice versa